```
// ==================================== 文件: rust/compute/crates/core/src/lib.rs ====================================
//! # compute-core
//!
//! 体素化切削仿真的纯 Rust 核心库。
//!
//! 提供：
//! - [`voxel_grid`]：3D 体素网格，紧凑位存储 + 高速批量运算。
//! - [`tool`]：6 种刀具几何的解析体素化（球头/平底/圆角平底/锥度/球头锥度/成形）。
//! - [`cutting`]：批量切削核心，SIMD 风格块运算 + 边界裁剪。
//! - [`error`]：统一错误类型，所有 API 不 panic。
//!
//! 所有函数都遵循 **Result 返回** 风格；不依赖任何 Python 运行时。
//!
//! ## 复杂度
//!
//! `apply_tool_mask_batch` 的时间复杂度为 `O(P * M)`，其中 `P` 为刀位点数、`M` 为
//! 刀具掩码体素数（`≤ (2*r/voxel_size)³`）。相比原 Python 三重循环
//! `O(N * V * T)`，去除了对全部 `V` 个体素的线性扫描，单帧复杂度从 `O(V)`
//! 降为 `O(M)`（`M << V`）。

#![deny(rust_2018_idioms)]
#![warn(missing_debug_implementations)]

pub mod cutting;
pub mod error;
pub mod tool;
pub mod voxel_grid;

pub use crate::cutting::{apply_tool_mask_batch, discretize_linear_segment, BatchResult};
pub use crate::error::{ComputeError, ComputeResult};
pub use crate::tool::{build_tool_mask, ToolGeometry, ToolType};
pub use crate::voxel_grid::{VoxelGrid, VoxelGridShape};
// ==================================== 文件: rust/compute/crates/pyo3_bindings/src/lib.rs ====================================
//! PyO3 绑定层：将 [`compute_core`] 中的核心算法暴露为 Python 可调用模块。
//!
//! ## 模块结构
//!
//! - `compute._native`: 顶级 Python 模块（maturin 注册）。
//!   - 子模块 `voxel_cutter`: 体素化切削函数。
//!
//! ## 零拷贝策略
//!
//! - `apply_tool_mask` 直接消费 `numpy.ndarray` 的位/字节视图；
//!   切削完成后通过 `unsafe { array.as_array_mut() }` 写回原数组。
//! - 任何错误都通过 `PyErr` 上抛，不 panic。
//!
//! ## 公开 API
//!
//! ```python
//! from compute import voxel_cutter as vc
```

```
//!
//! removed = vc.apply_tool_mask(
//!     grid_view,            # numpy bool 数组 (nx, ny, nz)
//!     tool_mask_view,       # numpy bool 数组 (mx, my, mz)
//!     points,               # numpy float64 (N, 3)
//!     bbox_min,             # (x0, y0, z0)
//!     voxel_size,           # float
//!     padding,              # float
//! )
//! mask = vc.build_tool_mask(
//!     tool_type="ball",
//!     diameter=10.0,
//!     corner_radius=5.0,
//!     cutting_length=50.0,
//!     voxel_size=1.0,
//!     taper_angle_deg=0.0,
//!     form_profile=None,
//! )
//! ```

use std::panic::{catch_unwind, AssertUnwindSafe};

use numpy::PyArrayMethods;
use pyo3::exceptions::{PyRuntimeError, PyValueError};
use pyo3::prelude::*;
use pyo3::types::{PyDict, PyList, PyModule, PyTuple};

use compute_core::{
    apply_tool_mask_batch, build_tool_mask, discretize_linear_segment, BatchResult, ToolGeometry,
    ToolType, VoxelGrid, VoxelGridShape,
};

/// 顶层 `compute._native` 模块。
#[pymodule]
fn _native(_py: Python<'_>, m: &Bound<'_, PyModule>) -> PyResult<()> {
    m.add("__version__", env!("CARGO_PKG_VERSION"))?;
    m.add("compute_core_version", compute_core::compute_core_version())?;
    m.add_function(wrap_pyfunction!(is_available, m)?)?;
    let vc = PyModule::new_bound(_py, "voxel_cutter")?;
    vc.add_function(wrap_pyfunction!(vc_apply_tool_mask, &vc)?)?;
    vc.add_function(wrap_pyfunction!(vc_build_tool_mask, &vc)?)?;
    vc.add_function(wrap_pyfunction!(vc_discretize_linear_segment, &vc)?)?;
    vc.add_function(wrap_pyfunction!(vc_benchmark_mask_cut, &vc)?)?;
    m.add_submodule(&vc)?;
    Ok(())
}

/// 暴露给 Python 的版本号 / 编译信息。
pub fn compute_core_version() -> &'static str {
    env!("CARGO_PKG_VERSION")
```

```
}

#[pyfunction]
fn is_available() -> bool {
    true
}

// ============================================================================
// voxel_cutter 子模块函数
// ============================================================================

/// 批量应用刀具掩码到体素网格。
///
/// # 参数
/// - `grid`: `numpy.ndarray`, `dtype=bool`, 形状 `(nx, ny, nz)`。
/// - `tool_mask`: `numpy.ndarray`, `dtype=bool`, 形状 `(mx, my, mz)`。
/// - `points`: `numpy.ndarray`, `dtype=float64`, 形状 `(N, 3)`。
/// - `bbox_min`: 长度 3 的可迭代对象。
/// - `voxel_size`: `float`。
/// - `padding`: `float`。
///
/// # 返回
/// `dict`: `{"removed": int, "skipped": int, "points": int}`
#[pyfunction]
#[pyo3(signature = (grid, tool_mask, points, bbox_min, voxel_size, padding))]
fn vc_apply_tool_mask(
    py: Python<'_>,
    grid: &Bound<'_, pyo3::PyAny>,
    tool_mask: &Bound<'_, pyo3::PyAny>,
    points: &Bound<'_, pyo3::PyAny>,
    bbox_min: &Bound<'_, pyo3::PyAny>,
    voxel_size: f64,
    padding: f64,
) -> PyResult<Py<PyDict>> {
    with_panic_guard(|| {
        // ---- 1. 校验与提取输入 ----
        let grid_arr: numpy::PyReadwriteArray<'_, bool, numpy::Ix3> =
            grid.extract().map_err(|e: pyo3::PyErr| {
                PyValueError::new_err(format!("grid must be bool ndarray (nx,ny,nz): {e}"))
            })?;
        let tool_arr: numpy::PyReadonlyArray<'_, bool, numpy::Ix3> =
            tool_mask.extract().map_err(|e: pyo3::PyErr| {
                PyValueError::new_err(format!("tool_mask must be bool ndarray (mx,my,mz): {e}"))
            })?;
        let pts_arr: numpy::PyReadonlyArray<'_, f64, numpy::Ix2> =
            points.extract().map_err(|e: pyo3::PyErr| {
                PyValueError::new_err(format!("points must be float64 ndarray (N,3): {e}"))
            })?;

        let bbox = extract_vec3(bbox_min)?;
```

```
        let g_shape_arr = grid_arr.shape();
        let nx = g_shape_arr[0];
        let ny = g_shape_arr[1];
        let nz = g_shape_arr[2];
        let shape = VoxelGridShape::new(nx, ny, nz)
            .map_err(|e| PyValueError::new_err(format!("invalid grid shape: {e}")))?;

        // 将 grid 复制到我们的 VoxelGrid（避免直接持有 numpy 缓冲；体素规模可控）
        let mut voxel_grid = VoxelGrid::new(shape);
        unsafe {
            let g_view = grid_arr.as_array();
            for ((x, y, z), &v) in g_view.indexed_iter() {
                if v {
                    voxel_grid.set(x, y, z, true);
                }
            }
        }

        let t_shape_arr = tool_arr.shape();
        let tnx = t_shape_arr[0];
        let tny = t_shape_arr[1];
        let tnz = t_shape_arr[2];
        let tool_shape = VoxelGridShape::new(tnx, tny, tnz)
            .map_err(|e| PyValueError::new_err(format!("invalid tool_mask shape: {e}")))?;

        let mut tool_bits = vec![0u64; tool_shape.word_count()];
        unsafe {
            let t_view = tool_arr.as_array();
            for ((x, y, z), &v) in t_view.indexed_iter() {
                if v {
                    let linear = tool_shape.xyz_to_linear(x, y, z);
                    tool_bits[linear >> 6] |= 1u64 << (linear & 63);
                }
            }
        }

        // points
        let pts_view = unsafe { pts_arr.as_array() };
        let n = pts_view.shape()[0];
        let mut points_flat = Vec::with_capacity(n * 3);
        for row in pts_view.rows() {
            points_flat.push(row[0]);
            points_flat.push(row[1]);
            points_flat.push(row[2]);
        }

        // ---- 2. 释放 GIL，执行核心算法 ----
        let result: BatchResult = py.allow_threads(|| {
            apply_tool_mask_batch(
                &mut voxel_grid,
```

```
                tool_shape,
                &tool_bits,
                &points_flat,
                bbox,
                voxel_size,
                padding,
            )
        })
        .map_err(|e| PyRuntimeError::new_err(format!("apply_tool_mask_batch failed: {e}")))?;

        // ---- 3. 将结果写回 grid（就地修改）----
        unsafe {
            let mut g_view = grid_arr.as_array_mut();
            for ((x, y, z), v) in g_view.indexed_iter_mut() {
                *v = voxel_grid.get(x, y, z);
            }
        }

        // ---- 4. 返回结果字典 ----
        let dict = PyDict::new_bound(py);
        dict.set_item("removed", result.removed as i64)?;
        dict.set_item("skipped", result.skipped as i64)?;
        dict.set_item("points", result.points as i64)?;
        Ok(dict.unbind())
    })
}

/// 构造刀具掩码。
///
/// # 参数
/// - `tool_type`: `"ball" | "flat" | "bullnose" | "tapered" | "balltapered" | "form"`
/// - `diameter`: `float`, 刀具直径
/// - `corner_radius`: `float`, 圆角半径
/// - `cutting_length`: `float`, 切削刃长
/// - `voxel_size`: `float`, 体素边长
/// - `taper_angle_deg`: `float`, 仅锥度/球头锥度使用
/// - `form_profile`: `list[tuple[float, float]] | None`, 仅 `form` 使用
///
/// # 返回
/// `(numpy.ndarray(dtype=bool, shape=(n,n,n)), info_dict)`:
///   - 第一个元素是 3D 刀具掩码（**零拷贝**: `shape` 与 `bits` 来自 Rust 端）
///   - 第二个元素是元数据字典
#[pyfunction]
#[pyo3(signature = (tool_type, diameter, corner_radius, cutting_length, voxel_size, taper_angle_deg=0.0, form_profile=None))]
fn vc_build_tool_mask<'py>(
    py: Python<'py>,
    tool_type: &str,
    diameter: f64,
    corner_radius: f64,
    cutting_length: f64,
```

```
    voxel_size: f64,
    taper_angle_deg: f64,
    form_profile: Option<&Bound<'_, PyAny>>,
) -> PyResult<(Bound<'py, numpy::PyArray<bool, numpy::Ix3>>, Bound<'py, PyDict>)> {
    with_panic_guard(|| {
        let tt = ToolType::parse(tool_type)
            .map_err(|e| PyValueError::new_err(format!("invalid tool_type: {e}")))?;

        // 成形轮廓：转换成 Vec 存储到 arena 中
        let arena: Vec<(f64, f64)> = if let Some(fp) = form_profile {
            if matches!(tt, ToolType::Form) {
                extract_form_profile(fp)?
            } else {
                Vec::new()
            }
        } else {
            Vec::new()
        };

        let geom = ToolGeometry {
            tool_type: tt,
            diameter,
            corner_radius,
            cutting_length,
            taper_angle_deg,
            form_profile: if arena.is_empty() {
                &[]
            } else {
                arena.as_slice()
            },
        };
        let (shape, bits) = build_tool_mask(&geom, voxel_size)
            .map_err(|e| PyValueError::new_err(format!("build_tool_mask failed: {e}")))?;

        // 把位图展开为 flat bool 数组（n×n×n）
        let total = shape.total();
        let mut flat = vec![false; total];
        for (word_idx, &word) in bits.iter().enumerate() {
            for bit in 0..64 {
                if (word >> bit) & 1 == 1 {
                    let linear = word_idx * 64 + bit;
                    if linear < total {
                        flat[linear] = true;
                    }
                }
            }
        }
        let array = numpy::PyArray::from_vec_bound(py, flat)
            .reshape([shape.nx, shape.ny, shape.nz])
            .map_err(|e| PyValueError::new_err(format!("reshape failed: {e}")))?;
```

```
        // 元数据
        let set_bits: usize = bits.iter().map(|w| w.count_ones() as usize).sum();
        let info = PyDict::new_bound(py);
        info.set_item("nx", shape.nx)?;
        info.set_item("ny", shape.ny)?;
        info.set_item("nz", shape.nz)?;
        info.set_item("voxel_size", voxel_size)?;
        info.set_item("set_bits", set_bits as i64)?;
        info.set_item("tool_type", tool_type)?;
        info.set_item("diameter", diameter)?;
        info.set_item("corner_radius", corner_radius)?;
        info.set_item("cutting_length", cutting_length)?;
        info.set_item("taper_angle_deg", taper_angle_deg)?;
        Ok((array, info))
    })
}

/// 离散化直线路径。
#[pyfunction]
fn vc_discretize_linear_segment(
    py: Python<'_>,
    start: (f64, f64, f64),
    end: (f64, f64, f64),
    step: f64,
) -> PyResult<Bound<'_, numpy::PyArray<f64, numpy::Ix2>>> {
    with_panic_guard(|| {
        let s = [start.0, start.1, start.2];
        let e = [end.0, end.1, end.2];
        let flat: Vec<f64> = py.allow_threads(|| discretize_linear_segment(s, e, step));
        let n = flat.len() / 3;
        let arr = numpy::PyArray::from_vec_bound(py, flat)
            .reshape([n, 3])
            .map_err(|e| PyValueError::new_err(format!("reshape failed: {e}")))?;
        Ok(arr)
    })
}

/// 端到端基准测试入口（用于 Python 端性能测试与 Rust 单测交叉验证）。
///
/// # 参数
/// - `grid_size`: 网格单边长度（立方体）。
/// - `diameter`: 刀具直径。
/// - `voxel_size`: 体素边长。
/// - `num_points`: 刀位点数。
///
/// # 返回
/// `dict`：`{elapsed_ms, removed, points}`
#[pyfunction]
#[pyo3(signature = (grid_size=100, diameter=8.0, voxel_size=1.0, num_points=100))]
```

```
fn vc_benchmark_mask_cut<'py>(
    py: Python<'py>,
    grid_size: usize,
    diameter: f64,
    voxel_size: f64,
    num_points: usize,
) -> PyResult<Bound<'py, PyDict>> {
    with_panic_guard(|| {
        if grid_size == 0 || num_points == 0 {
            return Err(PyValueError::new_err("grid_size and num_points must be > 0"));
        }
        // 1. 构建全实心网格
        let shape = VoxelGridShape::new(grid_size, grid_size, grid_size)
            .map_err(|e| PyValueError::new_err(format!("shape: {e}")))?;
        let mut grid = VoxelGrid::new(shape);
        for i in 0..shape.total() {
            grid.set_linear(i, true);
        }
        // 2. 构建刀具掩码
        let geom = ToolGeometry::new(ToolType::Flat, diameter, 0.0, diameter);
        let (t_shape, t_bits) = build_tool_mask(&geom, voxel_size)
            .map_err(|e| PyValueError::new_err(format!("mask: {e}")))?;
        // 3. 构造刀位点：均匀分布于网格中心区域
        let mut points = Vec::with_capacity(num_points * 3);
        let stride = (grid_size as f64) / ((num_points as f64).sqrt() + 1.0);
        let mut i = 0;
        let mut x = stride;
        while x < grid_size as f64 && i < num_points {
            let mut y = stride;
            while y < grid_size as f64 && i < num_points {
                points.push(x);
                points.push(y);
                points.push(grid_size as f64 * 0.5);
                y += stride;
                i += 1;
            }
            x += stride;
        }
        // 4. 计时
        let start_t = std::time::Instant::now();
        let result = py.allow_threads(|| {
            apply_tool_mask_batch(
                &mut grid,
                t_shape,
                &t_bits,
                &points,
                [0.0, 0.0, 0.0],
                voxel_size,
                0.0,
            )
```

```
        })
        .map_err(|e| PyRuntimeError::new_err(format!("apply failed: {e}")))?;
        let elapsed = start_t.elapsed();

        let dict = PyDict::new_bound(py);
        dict.set_item("elapsed_ms", elapsed.as_secs_f64() * 1000.0)?;
        dict.set_item("removed", result.removed as i64)?;
        dict.set_item("points", result.points as i64)?;
        dict.set_item("grid_voxels_remaining", grid.count_true() as i64)?;
        Ok(dict)
    })
}

// ============================================================================
// 工具函数
// ============================================================================

/// 提取 `bbox_min` 三元组。
fn extract_vec3(obj: &Bound<'_, pyo3::PyAny>) -> PyResult<[f64; 3]> {
    // 支持 (x, y, z) tuple、list、numpy 数组
    if let Ok(t) = obj.extract::<(f64, f64, f64)>() {
        return Ok([t.0, t.1, t.2]);
    }
    if let Ok(l) = obj.downcast::<PyList>() {
        if l.len() == 3 {
            return Ok([
                l.get_item(0)?.extract()?,
                l.get_item(1)?.extract()?,
                l.get_item(2)?.extract()?,
            ]);
        }
    }
    if let Ok(t) = obj.downcast::<PyTuple>() {
        if t.len() == 3 {
            return Ok([
                t.get_item(0)?.extract()?,
                t.get_item(1)?.extract()?,
                t.get_item(2)?.extract()?,
            ]);
        }
    }
    Err(PyValueError::new_err("bbox_min must be a 3-element sequence"))
}

/// 提取成形轮廓（list of (z, r) tuples）。
fn extract_form_profile(obj: &Bound<'_, pyo3::PyAny>) -> PyResult<Vec<(f64, f64)>> {
    let mut out = Vec::new();
    if let Ok(seq) = obj.extract::<Vec<(f64, f64)>>() {
        return Ok(seq);
    }
```

```
    if let Ok(l) = obj.downcast::<PyList>() {
        for item in l.iter() {
            if let Ok(t) = item.extract::<(f64, f64)>() {
                out.push(t);
            } else {
                return Err(PyValueError::new_err(
                    "form_profile items must be (z, r) tuples",
                ));
            }
        }
        return Ok(out);
    }
    Err(PyValueError::new_err(
        "form_profile must be a list of (z, r) tuples",
    ))
}

/// panic 安全网：将内部 panic 转换为 `PyRuntimeError`。
fn with_panic_guard<F, R>(f: F) -> PyResult<R>
where
    F: FnOnce() -> PyResult<R> + UnwindSafe,
{
    match catch_unwind(AssertUnwindSafe(f)) {
        Ok(r) => r,
        Err(e) => {
            let msg = if let Some(s) = e.downcast_ref::<String>() {
                s.clone()
            } else if let Some(s) = e.downcast_ref::<&str>() {
                (*s).to_string()
            } else {
                "unknown panic in compute engine".to_string()
            };
            Err(PyRuntimeError::new_err(format!(
                "Rust compute engine panicked: {msg}"
            )))
        }
    }
}

// Trait alias to allow AssertUnwindSafe on closures
use std::panic::UnwindSafe;
// ================================== 文件: rust/compute/crates/core/benches/cutting_bench.rs ================================…
//! 体素切削核心算法基准测试。
//!
//! 使用 `cargo bench` 运行。
//! 典型输出：100×100×100 网格、100 刀位点、voxel=1.0、刀具直径=8mm。

use criterion::{criterion_group, criterion_main, BenchmarkId, Criterion};
use compute_core::{
    apply_tool_mask_batch, build_tool_mask, discretize_linear_segment, ToolGeometry, ToolType,
```

```
    VoxelGrid, VoxelGridShape,
};

fn build_solid_grid(n: usize) -> VoxelGrid {
    let shape = VoxelGridShape::new(n, n, n).unwrap();
    let mut g = VoxelGrid::new(shape);
    for i in 0..shape.total() {
        g.set_linear(i, true);
    }
    g
}

fn build_horizontal_line_points(n: usize) -> Vec<f64> {
    let mut pts = Vec::with_capacity(n * 3);
    let stride = (n as f64) / ((n as f64).sqrt() + 1.0);
    let mut k = 0;
    let mut x = stride;
    while x < n as f64 && k < n {
        let mut y = stride;
        while y < n as f64 && k < n {
            pts.push(x);
            pts.push(y);
            pts.push(n as f64 * 0.5);
            y += stride;
            k += 1;
        }
        x += stride;
    }
    pts
}

fn bench_mask_cut(c: &mut Criterion) {
    let mut group = c.benchmark_group("mask_cut");
    for &grid_size in &[10usize, 30, 50] {
        let mut grid = build_solid_grid(grid_size);
        let geom = ToolGeometry::new(ToolType::Flat, 8.0, 0.0, 8.0);
        let (t_shape, t_bits) = build_tool_mask(&geom, 1.0).unwrap();
        let pts = build_horizontal_line_points(50);

        group.bench_with_input(
            BenchmarkId::from_parameter(grid_size),
            &grid_size,
            |b, _| {
                b.iter(|| {
                    let mut g = grid.clone();
                    let r = apply_tool_mask_batch(
                        &mut g,
                        t_shape,
                        &t_bits,
                        &pts,
```

```
                        [0.0, 0.0, 0.0],
                        1.0,
                        0.0,
                    )
                    .unwrap();
                    criterion::black_box(r.removed);
                });
            },
        );
    }
    group.finish();
}

fn bench_build_mask(c: &mut Criterion) {
    let mut group = c.benchmark_group("build_mask");
    for &diameter in &[4.0f64, 8.0, 16.0] {
        group.bench_with_input(
            BenchmarkId::from_parameter(diameter),
            &diameter,
            |b, &d| {
                let geom = ToolGeometry::new(ToolType::Ball, d, d * 0.5, d * 5.0);
                b.iter(|| {
                    let (s, bits) = build_tool_mask(&geom, 0.5).unwrap();
                    criterion::black_box((s.total(), bits.len()));
                });
            },
        );
    }
    group.finish();
}

fn bench_discretize(c: &mut Criterion) {
    c.bench_function("discretize_long_line", |b| {
        b.iter(|| {
            let pts = discretize_linear_segment([0.0, 0.0, 0.0], [1000.0, 1000.0, 0.0], 0.5);
            criterion::black_box(pts.len());
        });
    });
}

criterion_group!(benches, bench_mask_cut, bench_build_mask, bench_discretize);
criterion_main!(benches);
// ================================== 文件: rust/compute/crates/core/src/cutting.rs ==================================
//! 切削仿真核心算法。
//!
//! 提供：
//! - [`apply_tool_mask_batch`]：批量应用刀具掩码到体素网格。
//! - [`discretize_linear_segment`]：将直线路径离散为等间距采样点。
//! - [`BatchResult`]：批量切削的统计结果。
//!
```

```
//! ## 性能
//!
//! 关键优化：
//! 1. **位运算裁剪** —— 切削操作在位图层面完成；
//!    当网格与掩码均按 `u64` 字对齐时，可使用 `mask_word &= tool_word` 一次完成 64 体的批量清除。
//! 2. **AABB 预过滤** —— 远在 `O(N)` 之外的刀位点通过单点比较提前 skip；
//!    距离裁剪仅做 `O(1)` 的整数比较，避免进入子区域循环。
//! 3. **行主序** —— 内存访问顺序与底层位图存储一致，硬件预取器友好。
//! 4. **零分配** —— 不使用 `Vec::new()` 在内层循环；预计算所有边界索引。

use crate::error::{ComputeError, ComputeResult};
use crate::voxel_grid::{VoxelGrid, VoxelGridShape};

/// 批量切削结果统计。
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub struct BatchResult {
    /// 处理的刀位点数。
    pub points: usize,
    /// 切除的体素总数。
    pub removed: usize,
    /// 因越界被跳过的刀位点数。
    pub skipped: usize,
}

impl BatchResult {
    pub fn merge(&mut self, other: BatchResult) {
        self.points += other.points;
        self.removed += other.removed;
        self.skipped += other.skipped;
    }
}

/// 将刀具掩码（`u64` 位图）批量应用到体素网格上。
///
/// # 参数
/// - `grid`: 工件体素网格（就地修改）。
/// - `tool_shape`: 刀具掩码形状。
/// - `tool_bits`: 刀具掩码位图（与 `grid.bits()` 同样行主序展平）。
/// - `points`: `(N, 3)` 刀位点坐标数组，连续存储。
/// - `bbox_min`: 工件包围盒最小点 `(x0, y0, z0)`。
/// - `voxel_size`: 体素边长。
/// - `padding`: 工件包围盒外扩量（与 Python 端语义一致）。
///
/// # 返回
/// - [`BatchResult`] 包含处理的刀位点数、切除体素数与越界跳过的刀位点数。
///
/// # 错误
/// - 若 `points.len() % 3 != 0` 返回 `ShapeMismatch`。
/// - 若 `tool_bits.len() != tool_shape.word_count()` 返回 `ShapeMismatch`。
pub fn apply_tool_mask_batch(
```

```
    grid: &mut VoxelGrid,
    tool_shape: VoxelGridShape,
    tool_bits: &[u64],
    points: &[f64],
    bbox_min: [f64; 3],
    voxel_size: f64,
    padding: f64,
) -> ComputeResult<BatchResult> {
    if voxel_size <= 0.0 || !voxel_size.is_finite() {
        return Err(ComputeError::InvalidVoxelSize { voxel_size });
    }
    if points.len() % 3 != 0 {
        return Err(ComputeError::ShapeMismatch {
            expected: "points.len() % 3 == 0".to_string(),
            actual: format!("points.len() = {}", points.len()),
        });
    }
    let expected_tool_words = tool_shape.word_count();
    if tool_bits.len() != expected_tool_words {
        return Err(ComputeError::ShapeMismatch {
            expected: format!("tool_bits len = {}", expected_tool_words),
            actual: format!("tool_bits len = {}", tool_bits.len()),
        });
    }

    let n_points = points.len() / 3;
    let mut result = BatchResult {
        points: n_points,
        ..Default::default()
    };

    let (gx, gy, gz) = (grid.shape().nx, grid.shape().ny, grid.shape().nz);
    let (tx, ty, tz) = (tool_shape.nx, tool_shape.ny, tool_shape.nz);
    let hx = tx / 2;
    let hy = ty / 2;
    let hz = tz / 2;

    let inv_vs = 1.0 / voxel_size;
    let mut removed_total = 0usize;
    let mut skipped_total = 0usize;

    for i in 0..n_points {
        let x = points[i * 3];
        let y = points[i * 3 + 1];
        let z = points[i * 3 + 2];

        // 坐标转网格索引（带 padding），等价于 Python 端：
        //   np.round(([x,y,z] - bbox_min + padding) / voxel_size).astype(int)
        let tip = [
            ((x - bbox_min[0] + padding) * inv_vs).round() as i64,
```

```
            ((y - bbox_min[1] + padding) * inv_vs).round() as i64,
            ((z - bbox_min[2] + padding) * inv_vs).round() as i64,
        ];

        // AABB 预过滤：刀具覆盖范围与网格求交
        let gx_min = (tip[0] - hx as i64).max(0) as usize;
        let gy_min = (tip[1] - hy as i64).max(0) as usize;
        let gz_min = (tip[2] - hz as i64).max(0) as usize;
        let gx_max = (tip[0] + hx as i64 + 1).min(gx as i64).max(0) as usize;
        let gy_max = (tip[1] + hy as i64 + 1).min(gy as i64).max(0) as usize;
        let gz_max = (tip[2] + hz as i64 + 1).min(gz as i64).max(0) as usize;

        if gx_min >= gx_max || gy_min >= gy_max || gz_min >= gz_max {
            skipped_total += 1;
            continue;
        }

        let mx_start = gx_min as i64 - (tip[0] - hx as i64);
        let my_start = gy_min as i64 - (tip[1] - hy as i64);
        let mz_start = gz_min as i64 - (tip[2] - hz as i64);
        let mx_end = mx_start + (gx_max - gx_min) as i64;
        let my_end = my_start + (gy_max - gy_min) as i64;
        let mz_end = mz_start + (gz_max - gz_min) as i64;

        removed_total += apply_subregion(
            grid,
            tool_shape,
            tool_bits,
            gx_min,
            gy_min,
            gz_min,
            gx_max,
            gy_max,
            gz_max,
            mx_start,
            my_start,
            mz_start,
            mx_end,
            my_end,
            mz_end,
        );
    }

    result.removed = removed_total;
    result.skipped = skipped_total;
    Ok(result)
}

#[allow(clippy::too_many_arguments)]
fn apply_subregion(
```

```
    grid: &mut VoxelGrid,
    tool_shape: VoxelGridShape,
    tool_bits: &[u64],
    gx_min: usize,
    gy_min: usize,
    gz_min: usize,
    gx_max: usize,
    gy_max: usize,
    gz_max: usize,
    mx_start: i64,
    my_start: i64,
    mz_start: i64,
    mx_end: i64,
    my_end: i64,
    mz_end: i64,
) -> usize {
    let mut removed = 0usize;
    let ny = grid.shape().ny;
    let nz = grid.shape().nz;
    let tny = tool_shape.ny;
    let tnz = tool_shape.nz;
    let grid_bits = grid.bits_mut();

    // 逐行扫描：内层以 (x, y, z) 三层循环
    for gx in gx_min..gx_max {
        let mx = mx_start + (gx - gx_min) as i64;
        if mx < 0 || mx >= tool_shape.nx as i64 {
            continue;
        }
        for gy in gy_min..gy_max {
            let my = my_start + (gy - gy_min) as i64;
            if my < 0 || my >= tny as i64 {
                continue;
            }
            // 计算每行基址（线性索引）
            let grid_row_base = (gx * ny + gy) * nz;
            let tool_row_base = (mx as usize * tny + my as usize) * tnz;
            for gz in gz_min..gz_max {
                let mz = mz_start + (gz - gz_min) as i64;
                if mz < 0 || mz >= tnz as i64 {
                    continue;
                }
                let grid_idx = grid_row_base + gz;
                let tool_idx = tool_row_base + mz as usize;

                let g_word = grid_idx >> 6;
                let g_bit = grid_idx & 63;
                let t_word = tool_idx >> 6;
                let t_bit = tool_idx & 63;
```

```
                let tool_bit = (tool_bits[t_word] >> t_bit) & 1;
                if tool_bit == 0 {
                    continue;
                }
                let g_set = (grid_bits[g_word] >> g_bit) & 1;
                if g_set == 1 {
                    grid_bits[g_word] &= !(1u64 << g_bit);
                    removed += 1;
                }
            }
        }
    }
    removed
}

/// 将直线路径 `start → end` 离散为等间距采样点。
///
/// # 参数
/// - `start`, `end`: 起止点 (x, y, z)。
/// - `step`: 采样间距。
///
/// # 返回
/// 扁平 `(N*3)` 数组：`[x0, y0, z0, x1, y1, z1, ...]`。
pub fn discretize_linear_segment(start: [f64; 3], end: [f64; 3], step: f64) -> Vec<f64> {
    if step <= 0.0 {
        return vec![start[0], start[1], start[2], end[0], end[1], end[2]];
    }
    let dx = end[0] - start[0];
    let dy = end[1] - start[1];
    let dz = end[2] - start[2];
    let dist = (dx * dx + dy * dy + dz * dz).sqrt();
    if dist < 1e-12 {
        return vec![start[0], start[1], start[2]];
    }
    let n = (dist / step).ceil() as usize;
    let n = n.max(1);
    let mut out = Vec::with_capacity((n + 1) * 3);
    for k in 0..=n {
        let t = k as f64 / n as f64;
        out.push(start[0] + t * dx);
        out.push(start[1] + t * dy);
        out.push(start[2] + t * dz);
    }
    out
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::tool::{build_tool_mask, ToolGeometry, ToolType};
```

```
fn make_grid(n: usize) -> VoxelGrid {
    let shape = VoxelGridShape::new(n, n, n).unwrap();
    VoxelGrid::new(shape)
}

fn make_solid_grid(n: usize) -> VoxelGrid {
    let shape = VoxelGridShape::new(n, n, n).unwrap();
    // 全部置 true
    let mut g = VoxelGrid::new(shape);
    let total = shape.total();
    for i in 0..total {
        g.set_linear(i, true);
    }
    g
}

fn small_flat_mask() -> (VoxelGridShape, Vec<u64>) {
    // 直径 2mm，voxel=1mm → 掩码 3x3x3
    let geom = ToolGeometry::new(ToolType::Flat, 2.0, 0.0, 2.0);
    build_tool_mask(&geom, 1.0).unwrap()
}

#[test]
fn rejects_misaligned_points() {
    let mut g = make_grid(10);
    let (ts, tb) = small_flat_mask();
    let r = apply_tool_mask_batch(
        &mut g,
        ts,
        &tb,
        &[1.0, 2.0], // 长度不是 3 的倍数
        [0.0, 0.0, 0.0],
        1.0,
        0.0,
    );
    assert!(matches!(r, Err(ComputeError::ShapeMismatch { .. })));
}

#[test]
fn rejects_wrong_tool_bits_length() {
    let mut g = make_grid(10);
    let (ts, _tb) = small_flat_mask();
    let r = apply_tool_mask_batch(
        &mut g,
        ts,
        &[0u64; 999], // 长度不匹配
        &[],
        [0.0, 0.0, 0.0],
        1.0,
```

```
            0.0,
        );
        assert!(matches!(r, Err(ComputeError::ShapeMismatch { .. })));
    }

    #[test]
    fn no_points_no_change() {
        let mut g = make_solid_grid(10);
        let g_before = g.clone();
        let (ts, tb) = small_flat_mask();
        let r = apply_tool_mask_batch(
            &mut g,
            ts,
            &tb,
            &[],
            [0.0, 0.0, 0.0],
            1.0,
            0.0,
        )
        .unwrap();
        assert_eq!(r.points, 0);
        assert_eq!(r.removed, 0);
        assert_eq!(g.bits(), g_before.bits());
    }

    #[test]
    fn center_cut_removes_mask_volume() {
        // 5x5x5 实心网格，中心点 (2,2,2)，bbox_min=(0,0,0)，voxel=1, padding=0
        let mut g = make_solid_grid(5);
        let (ts, tb) = small_flat_mask();
        let result = apply_tool_mask_batch(
            &mut g,
            ts,
            &tb,
            &[2.0, 2.0, 2.0],
            [0.0, 0.0, 0.0],
            1.0,
            0.0,
        )
        .unwrap();
        assert_eq!(result.points, 1);
        assert!(result.removed > 0);
        // 网格剩余应严格小于全 125
        assert!(g.count_true() < 125);
    }

    #[test]
    fn out_of_bounds_point_is_skipped() {
        let mut g = make_solid_grid(5);
        let (ts, tb) = small_flat_mask();
```

```
        let result = apply_tool_mask_batch(
            &mut g,
            ts,
            &tb,
            &[100.0, 100.0, 100.0], // 远离网格
            [0.0, 0.0, 0.0],
            1.0,
            0.0,
        )
        .unwrap();
        assert_eq!(result.points, 1);
        assert_eq!(result.skipped, 1);
        assert_eq!(result.removed, 0);
        assert_eq!(g.count_true(), 125);
    }

    #[test]
    fn batch_result_merge() {
        let a = BatchResult {
            points: 5,
            removed: 10,
            skipped: 1,
        };
        let mut b = a;
        b.merge(a);
        assert_eq!(b.points, 10);
        assert_eq!(b.removed, 20);
        assert_eq!(b.skipped, 2);
    }

    #[test]
    fn discretize_linear_emits_endpoints() {
        let pts = discretize_linear_segment([0.0, 0.0, 0.0], [10.0, 0.0, 0.0], 2.0);
        // 距离=10, step=2 → 5 段 → 6 个点
        assert_eq!(pts.len(), 18);
        assert_eq!(&pts[0..3], &[0.0, 0.0, 0.0]);
        assert_eq!(&pts[15..18], &[10.0, 0.0, 0.0]);
    }

    #[test]
    fn discretize_zero_distance_returns_single_point() {
        let pts = discretize_linear_segment([1.0, 1.0, 1.0], [1.0, 1.0, 1.0], 0.5);
        assert_eq!(pts, vec![1.0, 1.0, 1.0]);
    }

    #[test]
    fn multiple_cuts_are_cumulative() {
        let mut g = make_solid_grid(10);
        let (ts, tb) = small_flat_mask();
        let pts = vec![3.0, 3.0, 3.0, 6.0, 6.0, 6.0];
```

```
        let result = apply_tool_mask_batch(
            &mut g,
            ts,
            &tb,
            &pts,
            [0.0, 0.0, 0.0],
            1.0,
            0.0,
        )
        .unwrap();
        assert_eq!(result.points, 2);
        assert!(result.removed > 0);
    }

    #[test]
    fn extreme_voxel_size_validation() {
        let mut g = make_solid_grid(10);
        let (ts, tb) = small_flat_mask();
        assert!(apply_tool_mask_batch(
            &mut g,
            ts,
            &tb,
            &[0.0, 0.0, 0.0],
            [0.0, 0.0, 0.0],
            0.0, // invalid voxel size
            0.0,
        )
        .is_err());
    }
}
// ==================================== 文件: rust/compute/crates/core/src/error.rs ====================================
//! 统一错误类型与 Result 别名。
//!
//! 所有对外 API 都返回 `Result<T, ComputeError>`，避免 panic 上浮污染上层调用栈。
//! 错误可通过 `thiserror` 自动派生 `Display` 与 `Error` trait。

use thiserror::Error;

/// 计算引擎统一错误类型。
#[derive(Debug, Error)]
pub enum ComputeError {
    /// 体素网格维度非法（任意维度为 0）。
    #[error("invalid voxel grid shape: dimensions must be > 0, got [{nx}, {ny}, {nz}]")]
    InvalidShape { nx: usize, ny: usize, nz: usize },

    /// 体素尺寸非法（≤ 0 或非有限）。
    #[error("invalid voxel_size: {voxel_size} (must be positive finite)")]
    InvalidVoxelSize { voxel_size: f64 },

    /// 工具几何参数非法。
```

```
    #[error("invalid tool geometry: {message}")]
    InvalidTool { message: String },

    /// 输入数组形状不匹配。
    #[error("shape mismatch: expected {expected}, got {actual}")]
    ShapeMismatch { expected: String, actual: String },

    /// 内部缓冲区容量不足（理论上不应触发；如出现表明上游数据流异常）。
    #[error("internal buffer overflow: required {required} > capacity {capacity}")]
    BufferOverflow { required: usize, capacity: usize },
}

/// Result 简写。
pub type ComputeResult<T> = Result<T, ComputeError>;
// ==================================== 文件: rust/compute/crates/core/src/tool.rs ====================================
//! 刀具几何模块。
//!
//! 支持 6 种刀具：
//! - [`ToolType::Ball`] 球头刀
//! - [`ToolType::Flat`] 平底刀
//! - [`ToolType::BullNose`] 圆角平底刀（带拐角圆角的平底刀）
//! - [`ToolType::Tapered`] 锥度刀（圆锥形）
//! - [`ToolType::BallTapered`] 球头锥度刀（圆锥 + 球头尖）
//! - [`ToolType::Form`] 特殊成形刀（自定义包络曲线）
//!
//! 全部以解析形式给出体素掩码；通过 [`build_tool_mask`] 入口构造。
//!
//! ## 局部坐标系
//!
//! 刀具局部坐标系：刀尖位于原点 `(0, 0, 0)`，**Z 轴正方向朝向工件表面**。
//! 有效工作区为 `z ∈ [-cutting_length, 0]`、径向 `√(x² + y²) ≤ r`。
//!
//! ## 输出
//!
//! 刀具掩码使用 [`VoxelGrid`] 表示，三维 `bool` 位图。
//! 中心位置 `(cx, cy, cz)` 对应刀尖。

use crate::error::{ComputeError, ComputeResult};
use crate::voxel_grid::VoxelGridShape;

/// 刀具类型枚举。
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub enum ToolType {
    /// 球头刀：完整半球形刀尖。
    Ball,
    /// 平底刀：纯圆柱。
    Flat,
    /// 圆角平底刀：圆柱 + 底部圆角。
    BullNose,
    /// 锥度刀：圆锥（圆台）。
```

```
    Tapered,
    /// 球头锥度刀：圆锥 + 半球形刀尖。
    BallTapered,
    /// 成形刀：自定义轮廓（线性插值）。
    Form,
}

impl ToolType {
    /// 从字符串解析（大小写不敏感），失败返回 `InvalidTool`。
    pub fn parse(s: &str) -> ComputeResult<Self> {
        match s.to_ascii_lowercase().as_str() {
            "ball" | "ballnose" | "ball_nose" => Ok(ToolType::Ball),
            "flat" | "flatend" | "flat_end" => Ok(ToolType::Flat),
            "bullnose" | "bull_nose" | "bull" | "corner_radius" => Ok(ToolType::BullNose),
            "tapered" | "taper" | "cone" => Ok(ToolType::Tapered),
            "balltapered" | "ball_tapered" | "taperedball" | "tapered_ball" => {
                Ok(ToolType::BallTapered)
            }
            "form" | "formed" | "profile" => Ok(ToolType::Form),
            other => Err(ComputeError::InvalidTool {
                message: format!(
                    "unknown tool_type '{}' (valid: ball, flat, bullnose, tapered, balltapered, form)",
                    other
                ),
            }),
        }
    }
}

/// 刀具几何参数。
#[derive(Debug, Clone, Copy, PartialEq)]
pub struct ToolGeometry {
    pub tool_type: ToolType,
    /// 刀具直径 (mm)。
    pub diameter: f64,
    /// 拐角圆角半径 (mm)。`Flat` 类型为 0，`Ball` 类型通常等于半径。
    pub corner_radius: f64,
    /// 切削刃长 (mm)。
    pub cutting_length: f64,
    /// 锥度半角（度）。仅 `Tapered`/`BallTapered` 使用。
    pub taper_angle_deg: f64,
    /// 成形轮廓半径序列（mm）。仅 `Form` 使用；至少 2 个采样点。
    pub form_profile: &'static [(f64, f64)], // (z_offset_from_tip, radius)
}

impl ToolGeometry {
    /// 构造基础几何参数（带默认 `taper_angle_deg = 0.0`、`form_profile = &[]`）。
    pub fn new(
        tool_type: ToolType,
        diameter: f64,
```

```
        corner_radius: f64,
        cutting_length: f64,
    ) -> Self {
        Self {
            tool_type,
            diameter,
            corner_radius,
            cutting_length,
            taper_angle_deg: 0.0,
            form_profile: &[],
        }
    }

    /// 锥度刀构造。
    pub fn tapered(diameter: f64, cutting_length: f64, taper_angle_deg: f64) -> Self {
        Self {
            tool_type: ToolType::Tapered,
            diameter,
            corner_radius: 0.0,
            cutting_length,
            taper_angle_deg,
            form_profile: &[],
        }
    }

    /// 球头锥度刀构造。
    pub fn ball_tapered(
        diameter: f64,
        corner_radius: f64,
        cutting_length: f64,
        taper_angle_deg: f64,
    ) -> Self {
        Self {
            tool_type: ToolType::BallTapered,
            diameter,
            corner_radius,
            cutting_length,
            taper_angle_deg,
            form_profile: &[],
        }
    }

    /// 成形刀构造。
    pub fn form(diameter: f64, cutting_length: f64, profile: &'static [(f64, f64)]) -> Self {
        Self {
            tool_type: ToolType::Form,
            diameter,
            corner_radius: 0.0,
            cutting_length,
            taper_angle_deg: 0.0,
```

```
            form_profile: profile,
        }
    }

    /// 校验参数合法性。
    pub fn validate(&self) -> ComputeResult<()> {
        if !self.diameter.is_finite() || self.diameter <= 0.0 {
            return Err(ComputeError::InvalidTool {
                message: format!("diameter must be positive finite, got {}", self.diameter),
            });
        }
        if !self.cutting_length.is_finite() || self.cutting_length <= 0.0 {
            return Err(ComputeError::InvalidTool {
                message: format!(
                    "cutting_length must be positive finite, got {}",
                    self.cutting_length
                ),
            });
        }
        if self.corner_radius < 0.0 || !self.corner_radius.is_finite() {
            return Err(ComputeError::InvalidTool {
                message: format!(
                    "corner_radius must be non-negative finite, got {}",
                    self.corner_radius
                ),
            });
        }
        if self.corner_radius > self.diameter {
            return Err(ComputeError::InvalidTool {
                message: format!(
                    "corner_radius ({}) cannot exceed diameter ({})",
                    self.corner_radius, self.diameter
                ),
            });
        }
        if matches!(self.tool_type, ToolType::Tapered | ToolType::BallTapered) {
            if self.taper_angle_deg <= 0.0 || self.taper_angle_deg >= 90.0 {
                return Err(ComputeError::InvalidTool {
                    message: format!(
                        "taper_angle_deg must be in (0, 90), got {}",
                        self.taper_angle_deg
                    ),
                });
            }
        }
        if matches!(self.tool_type, ToolType::Form) {
            if self.form_profile.len() < 2 {
                return Err(ComputeError::InvalidTool {
                    message: "form profile requires at least 2 sample points".to_string(),
                });
```

```
            }
        }
        Ok(())
    }
}

/// 根据刀具参数生成体素掩码（位存储）。
///
/// # 参数
/// - `geom`: 刀具几何参数（须先 [`ToolGeometry::validate`]）。
/// - `voxel_size`: 体素边长 (mm)，须为正有限值。
///
/// # 返回
/// - `(shape, bits)`：`shape` 是掩码的维度，`bits` 是 `u64` 位图（与 `VoxelGrid::bits()` 兼容）。
///
/// # 性能
/// - 时间复杂度：`O(M)`，其中 `M = nx*ny*nz` 为掩码体素数。
/// - 内存复杂度：`O(M/64)` 字节。
pub fn build_tool_mask(
    geom: &ToolGeometry,
    voxel_size: f64,
) -> ComputeResult<(VoxelGridShape, Vec<u64>)> {
    geom.validate()?;
    if !voxel_size.is_finite() || voxel_size <= 0.0 {
        return Err(ComputeError::InvalidVoxelSize { voxel_size });
    }

    // 选用工具最大半径作为网格半径，确保覆盖所有形状。
    let max_r = geom.diameter * 0.5;
    let grid_half = (max_r / voxel_size).ceil() as i32 + 1;
    let n = (2 * grid_half + 1) as usize;
    let shape = VoxelGridShape::new(n, n, n)?;

    // 仅在 (z ∈ [-cutting_length, 0]) 内生成有效位；
    // 但保持形状对称便于 PyO3 端做掩码卷积。
    let mut bits = vec![0u64; shape.word_count()];
    let total = shape.total();

    // 预计算成形刀轮廓查找表（z 升序，r 单调），供 Form 类型使用。
    // 出于 API 简洁考虑，Form 在调用方传入 `form_profile` 时是 `&'static [(f64, f64)]`。
    // 我们直接在内部把 (z, r) 转为闭式判定。

    for linear in 0..total {
        let (xi, yi, zi) = {
            let z = linear % shape.nz;
            let yz = linear / shape.nz;
            let y = yz % shape.ny;
            let x = yz / shape.ny;
            (x as i32, y as i32, z as i32)
        };
```

```
        let gx = (xi - grid_half) as f64 * voxel_size;
        let gy = (yi - grid_half) as f64 * voxel_size;
        let gz = (zi - grid_half) as f64 * voxel_size; // z 局部坐标，0 = 中心

        // 注意：上面 `grid_range` 的中心是刀尖，但我们 grid_half 是从 (max_r + 1) 算的，
        // 实际刀尖应在 (grid_half, grid_half, grid_half) — 即坐标 z=0。
        // 因此 `gz` 即为相对刀尖的 Z 偏移（0 = 刀尖，负值 = 刀尖下方即工件内部）。
        if !inside_geometry(geom, gx, gy, gz) {
            continue;
        }
        // 设置位
        let word = linear >> 6;
        let bit = linear & 63;
        bits[word] |= 1u64 << bit;
    }

    Ok((shape, bits))
}

/// 判断点 `(x, y, z)`（局部坐标，刀尖在原点）是否在刀具占据区域内。
///
/// 坐标系约定：刀尖在 `(0, 0, 0)`，Z 轴正方向指向工件表面（上方），
/// Z 负方向为刀具进入工件的方向。
fn inside_geometry(geom: &ToolGeometry, x: f64, y: f64, z: f64) -> bool {
    let r = geom.diameter * 0.5;
    let r_xy = (x * x + y * y).sqrt();

    // 工作 Z 范围：刀尖以下到 cutting_length 处。
    // z=0 -> 刀尖, z=-cutting_length -> 切削刃末端（深入工件方向）。
    if z > 0.0 || z < -geom.cutting_length {
        return false;
    }
    // 工作半径外直接拒绝（加速大量点）。
    if r_xy > r + 1e-9 {
        return false;
    }

    match geom.tool_type {
        ToolType::Ball => {
            // 完整半球 + 圆柱延伸
            let cr = geom.corner_radius.max(1e-6);
            if z >= -cr + 1e-9 {
                // 半球部分：以 (0, 0, -cr) 为球心
                let dz = z + cr;
                return r_xy * r_xy + dz * dz <= cr * cr + 1e-9;
            }
            // 圆柱部分：z <= -cr
            r_xy <= r + 1e-9
        }
        ToolType::Flat => {
```

```
            // 纯圆柱
            r_xy <= r + 1e-9
        }
        ToolType::BullNose => {
            // 拐角圆角 + 圆柱
            let cr = geom.corner_radius;
            if cr <= 1e-9 {
                return r_xy <= r + 1e-9;
            }
            if z >= -cr + 1e-9 {
                // 圆角区域：球心 (0, 0, -cr)
                let dz = z + cr;
                return r_xy * r_xy + dz * dz <= cr * cr + 1e-9;
            }
            r_xy <= r + 1e-9
        }
        ToolType::Tapered => {
            // 锥度刀：从刀尖向 z=-cutting_length 半径线性变化（刀尖=0, 末端=r）
            // 锥度半角 β；r(z) = -z * tan(β)  for z ∈ [-cutting_length, 0]
            let beta = geom.taper_angle_deg.to_radians();
            let tan_b = beta.tan();
            // 刀尖处 r(0)=0; z=-cutting_length 处 r=r_max
            // 强制末端不小于 0、不超过 r：
            let max_r_at_end = (geom.cutting_length * tan_b).min(r);
            if z.abs() < 1e-9 {
                return r_xy <= 1e-9;
            }
            let r_local = (-z) * tan_b;
            if r_local > max_r_at_end + 1e-9 {
                // 已越过末端 → 视为无材料（被夹持，不参与切削）
                return false;
            }
            r_xy <= r_local + 1e-9
        }
        ToolType::BallTapered => {
            // 锥度 + 半球尖
            let cr = geom.corner_radius.max(1e-6);
            if z >= -cr + 1e-9 {
                let dz = z + cr;
                return r_xy * r_xy + dz * dz <= cr * cr + 1e-9;
            }
            // 锥度延伸（z 越深半径越大）
            let beta = geom.taper_angle_deg.to_radians();
            let tan_b = beta.tan();
            let r_local = cr + (-z - cr).max(0.0) * tan_b;
            if r_local > r + 1e-9 {
                return false;
            }
            r_xy <= r_local + 1e-9
        }
```

```
        ToolType::Form => {
            // 成形刀：线性插值自定义轮廓 (z, r(z))
            // z ∈ [-cutting_length, 0]，profile 至少 2 个点。
            if geom.form_profile.is_empty() {
                return false;
            }
            // 寻找包围 z 的两个 profile 节点
            let prof = geom.form_profile;
            if z <= prof[0].0 {
                return r_xy <= prof[0].1 + 1e-9;
            }
            if z >= prof[prof.len() - 1].0 {
                return r_xy <= prof[prof.len() - 1].1 + 1e-9;
            }
            // 线性插值
            let mut r_at_z = prof[0].1;
            for w in prof.windows(2) {
                let (z0, r0) = w[0];
                let (z1, r1) = w[1];
                if z >= z0 && z <= z1 {
                    let t = if (z1 - z0).abs() < 1e-12 {
                        0.0
                    } else {
                        (z - z0) / (z1 - z0)
                    };
                    r_at_z = r0 + t * (r1 - r0);
                    break;
                }
            }
            r_xy <= r_at_z + 1e-9
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn ball(d: f64) -> ToolGeometry {
        ToolGeometry::new(ToolType::Ball, d, d * 0.5, 50.0)
    }
    fn flat(d: f64) -> ToolGeometry {
        ToolGeometry::new(ToolType::Flat, d, 0.0, 50.0)
    }

    #[test]
    fn parses_all_six_tool_types() {
        for (alias, expected) in [
            ("ball", ToolType::Ball),
            ("BallNose", ToolType::Ball),
```

```
            ("flat", ToolType::Flat),
            ("flatend", ToolType::Flat),
            ("bullnose", ToolType::BullNose),
            ("bull", ToolType::BullNose),
            ("tapered", ToolType::Tapered),
            ("taper", ToolType::Tapered),
            ("balltapered", ToolType::BallTapered),
            ("tapered_ball", ToolType::BallTapered),
            ("form", ToolType::Form),
            ("profile", ToolType::Form),
        ] {
            assert_eq!(ToolType::parse(alias).unwrap(), expected);
        }
        assert!(ToolType::parse("unknown_xyz").is_err());
    }

    #[test]
    fn validate_rejects_invalid_params() {
        let g = ToolGeometry::new(ToolType::Ball, -1.0, 0.0, 50.0);
        assert!(g.validate().is_err());
        let g = ToolGeometry::new(ToolType::Ball, 10.0, 0.0, -1.0);
        assert!(g.validate().is_err());
        let g = ToolGeometry::new(ToolType::Flat, 10.0, 11.0, 50.0);
        assert!(g.validate().is_err());
    }

    #[test]
    fn flat_mask_center_is_filled() {
        let g = flat(10.0);
        let (shape, bits) = build_tool_mask(&g, 1.0).unwrap();
        let total = shape.total();
        let set_bits: usize = bits.iter().map(|w| w.count_ones() as usize).sum();
        assert!(set_bits > 0);
        // 中心体素必须被占据
        let (cx, cy, cz) = (shape.nx / 2, shape.ny / 2, shape.nz / 2);
        let idx = (cx * shape.ny + cy) * shape.nz + cz;
        assert!(idx < total);
        let word = idx >> 6;
        let bit = idx & 63;
        assert_ne!(bits[word] & (1u64 << bit), 0);
    }

    #[test]
    fn ball_mask_hemisphere_at_tip() {
        let g = ball(10.0);
        let (shape, bits) = build_tool_mask(&g, 0.5).unwrap();
        // 刀尖半球应被填充：刀尖 (0,0,0) 一定在内部
        let (cx, cy, cz) = (shape.nx / 2, shape.ny / 2, shape.nz / 2);
        let idx = (cx * shape.ny + cy) * shape.nz + cz;
        let word = idx >> 6;
```

```
    id: 'export-gcode',
    name: t('uxDemo.cmdExportGCodeName'),
    description: t('uxDemo.cmdExportGCodeDesc'),
    category: t('uxDemo.categoryToolpath'),
    icon: 'Document',
    action: () => {
      ElMessage.success(t('uxDemo.msgExportGCodeTriggered'))
    }
  },
  {
    id: 'start-simulation',
    name: t('uxDemo.cmdStartSimulationName'),
    description: t('uxDemo.cmdStartSimulationDesc'),
    category: t('uxDemo.categorySimulation'),
    icon: 'VideoPlay',
    action: () => {
      ElMessage.success(t('uxDemo.msgStartSimulationTriggered'))
    }
  },
  {
    id: 'ai-feature-recognition',
    name: t('uxDemo.cmdAiFeatureName'),
    description: t('uxDemo.cmdAiFeatureDesc'),
    category: t('uxDemo.categoryAI'),
    icon: 'MagicStick',
    action: () => {
      ElMessage.success(t('uxDemo.msgAiFeatureTriggered'))
    }
  },
  {
    id: 'view-examples',
    name: t('uxDemo.cmdViewExamplesName'),
    description: t('uxDemo.cmdViewExamplesDesc'),
    category: t('uxDemo.categoryHelp'),
    icon: 'Collection',
    action: () => {
      ElMessage.success(t('uxDemo.msgViewExamplesTriggered'))
    }
  },
  {
    id: 'open-settings',
    name: t('uxDemo.cmdOpenSettingsName'),
    description: t('uxDemo.cmdOpenSettingsDesc'),
    category: t('uxDemo.categorySystem'),
    icon: 'Setting',
    action: () => {
      ElMessage.success(t('uxDemo.msgOpenSettingsTriggered'))
    }
  }
])
```

```
// 组件引用
const tourRef = ref<InstanceType<typeof Tour> | null>(null)
const commandPaletteRef = ref<{ open: () => void } | null>(null)

// 统计数据
const exampleCount = computed(() => exampleProjects.length)
const commandCount = computed(() => registeredCommands.value.length)

// 方法
function startTour() {
  tourRef.value?.start()
}

function openCommandPalette() {
  commandPaletteRef.value?.open()
}

function onTourStart() {
  // 引导流程开始
}

function onTourStepChange(_index: number) {
  // 引导步骤变化回调（由 Tour 子组件触发，无需额外处理）
}

function onTourFinish() {
  ElMessage.success(t('uxDemo.msgTourCompleted'))
}

function onTourSkip(index: number) {
  ElMessage.info(t('uxDemo.msgTourSkipped', { step: index + 1 }))
}

function onCommandExecute(_command: Command) {
  // 命令执行回调（由命令面板触发，无需额外处理）
}

// 生命周期
onMounted(() => {
  // UX 演示页面已加载
})
</script>

<style scoped lang="scss">
.ux-demo {
  padding: 20px;
  max-width: 1400px;
  margin: 0 auto;
```

```
  .demo-section {
    margin-bottom: 20px;

    .card-header {
      display: flex;
      justify-content: space-between;
      align-items: center;

      h3 {
        margin: 0;
        font-size: 18px;
        font-weight: 600;
      }

      .header-actions {
        display: flex;
        gap: 12px;
      }
    }

    .feature-list {
      margin-top: 20px;

      h4 {
        margin: 0 0 16px;
        font-size: 16px;
        font-weight: 600;
        color: var(--text-primary);
      }

      .feature-desc {
        margin-left: 12px;
        color: var(--text-secondary);
        font-size: 14px;
      }
    }
  }
}
</style>
// ==================================== 文件: engineering/src/views/UpdateCenter.vue ====================================
<script setup lang="ts">
import { ref, onMounted, computed } from 'vue'
import { useI18n } from 'vue-i18n'
import { formatSecondsTimestamp } from '@/utils/formatters'
import http from '@/utils/http'
import { API_CONFIG, buildApiPath } from '@/config/api'
import { extractErrorMessage } from '@/utils/error-handler'
import { useProjectStore } from '@/stores/project'
import type { TagType } from '@/utils/statusHelpers'
```

```
const { t } = useI18n()
const notifications = ref<NotificationItem[]>([])
const loading = ref(false)
const statusFilter = ref('pending')
const previewDialog = ref(false)
const previewData = ref<NotificationPreview | null>(null)
const projectStore = useProjectStore()

interface NotificationItem {
  notification_id: string
  title: string
  description: string
  priority: string
  status: string
  expected_impact?: Record<string, number | string>
  created_at: number
}

interface NotificationPreview {
  title: string
  description: string
  change_preview: unknown
  expected_impact: unknown
}

const filteredNotifications = computed(() => {
  if (statusFilter.value === 'all') return notifications.value
  return notifications.value.filter(n => n.status === statusFilter.value)
})

function priorityTag(priority: string): { text: string; type: TagType } {
  const map: Record<string, { text: string; type: TagType }> = {
    optional: { text: t('updateCenter.priorityOptional'), type: 'info' },
    recommended: { text: t('updateCenter.priorityRecommended'), type: 'primary' },
    critical: { text: t('updateCenter.priorityCritical'), type: 'danger' },
  }
  return map[priority] || { text: priority, type: 'info' }
}

function statusTag(status: string): { text: string; type: TagType } {
  const map: Record<string, { text: string; type: TagType }> = {
    pending: { text: t('updateCenter.statusPending'), type: 'warning' },
    applied: { text: t('updateCenter.statusApplied'), type: 'success' },
    dismissed: { text: t('updateCenter.statusDismissed'), type: 'info' },
  }
  return map[status] || { text: status, type: 'info' }
}

async function fetchNotifications() {
  loading.value = true
```

```
  try {
    const res = await http.get(buildApiPath(API_CONFIG.V1, `/templates/updates/${projectStore.projectId}`), {
      params: { status: statusFilter.value === 'all' ? undefined : statusFilter.value }
    })
    if (res.data.code === 'SUCCESS') notifications.value = res.data.data
  } catch (error) {
    console.warn(t('updateCenter.errorFetchFailed'), extractErrorMessage(error))
  } finally {
    loading.value = false
  }
}

async function applyUpdate(id: string) {
  try {
    await http.post(buildApiPath(API_CONFIG.V1, `/templates/updates/apply/${id}`))
    fetchNotifications()
  } catch (error) {
    console.warn(t('updateCenter.errorApplyFailed'), extractErrorMessage(error))
  }
}

async function dismissUpdate(id: string) {
  try {
    await http.post(buildApiPath(API_CONFIG.V1, `/templates/updates/dismiss/${id}`))
    fetchNotifications()
  } catch (error) {
    console.warn(t('updateCenter.errorDismissFailed'), extractErrorMessage(error))
  }
}

async function showPreview(id: string) {
  try {
    const res = await http.get(buildApiPath(API_CONFIG.V1, `/templates/updates/preview/${id}`))
    if (res.data.code === 'SUCCESS') {
      previewData.value = res.data.data
      previewDialog.value = true
    }
  } catch (error) {
    console.warn(t('updateCenter.errorPreviewFailed'), extractErrorMessage(error))
  }
}

onMounted(fetchNotifications)
</script>

<template>
  <div class="update-center-page">
    <el-card class="header-card">
      <div class="page-header">
        <h2>{{ t('updateCenter.pageTitle') }}</h2>
```

```
        <el-button
          type="primary"
          @click="fetchNotifications"
        >
          {{ t('updateCenter.btnRefresh') }}
        </el-button>
      </div>
    </el-card>

    <el-card class="filter-card">
      <el-radio-group
        v-model="statusFilter"
        @change="fetchNotifications"
      >
        <el-radio-button value="pending">
          {{ t('updateCenter.filterPending') }}
        </el-radio-button>
        <el-radio-button value="applied">
          {{ t('updateCenter.filterApplied') }}
        </el-radio-button>
        <el-radio-button value="dismissed">
          {{ t('updateCenter.filterDismissed') }}
        </el-radio-button>
        <el-radio-button value="all">
          {{ t('updateCenter.filterAll') }}
        </el-radio-button>
      </el-radio-group>
    </el-card>

    <div
      v-if="loading"
      class="loading"
    >
      {{ t('updateCenter.loading') }}
    </div>

    <el-card
      v-for="notif in filteredNotifications"
      :key="notif.notification_id"
      class="notif-card"
      shadow="hover"
    >
      <template #header>
        <div class="notif-header">
          <span class="notif-title">{{ notif.title }}</span>
          <div class="notif-tags">
            <el-tag
              :type="priorityTag(notif.priority).type"
              size="small"
            >
```

```
            {{ priorityTag(notif.priority).text }}
          </el-tag>
          <el-tag
            :type="statusTag(notif.status).type"
            size="small"
          >
            {{ statusTag(notif.status).text }}
          </el-tag>
        </div>
      </div>
    </template>
    <div class="notif-body">
      <p>{{ notif.description }}</p>
      <div
        v-if="notif.expected_impact && Object.keys(notif.expected_impact).length > 0"
        class="impact-section"
      >
        <h4>{{ t('updateCenter.expectedImpact') }}</h4>
        <div
          v-for="(val, key) in notif.expected_impact"
          :key="key"
          class="impact-item"
        >
          <span class="impact-label">{{ key }}:</span>
          <span class="impact-value">{{ typeof val === 'number' ? (val * 100).toFixed(1) + '%' : val }}</span>
        </div>
      </div>
      <div class="notif-time">
        {{ formatSecondsTimestamp(notif.created_at) }}
      </div>
    </div>
    <template #footer>
      <div class="notif-actions">
        <el-button
          size="small"
          @click="showPreview(notif.notification_id)"
        >
          {{ t('updateCenter.btnPreview') }}
        </el-button>
        <el-button
          v-if="notif.status === 'pending'"
          type="success"
          size="small"
          @click="applyUpdate(notif.notification_id)"
        >
          {{ t('updateCenter.btnApply') }}
        </el-button>
        <el-button
          v-if="notif.status === 'pending'"
          type="info"
```

```
            size="small"
            @click="dismissUpdate(notif.notification_id)"
          >
            {{ t('updateCenter.btnDismiss') }}
          </el-button>
        </div>
      </template>
    </el-card>

    <el-empty
      v-if="!loading && filteredNotifications.length === 0"
      :description="t('updateCenter.emptyNoNotifications')"
    />

    <el-dialog
      v-model="previewDialog"
      :title="t('updateCenter.dialogTitle')"
      width="600px"
    >
      <div v-if="previewData">
        <h3>{{ previewData.title }}</h3>
        <p>{{ previewData.description }}</p>
        <h4>{{ t('updateCenter.sectionChanges') }}</h4>
        <pre class="change-preview">{{ JSON.stringify(previewData.change_preview, null, 2) }}</pre>
        <h4>{{ t('updateCenter.sectionExpectedEffect') }}</h4>
        <pre class="impact-preview">{{ JSON.stringify(previewData.expected_impact, null, 2) }}</pre>
      </div>
    </el-dialog>
  </div>
</template>

<style scoped>
.update-center-page { padding: 20px; }
.page-header { display: flex; justify-content: space-between; align-items: center; }
.page-header h2 { margin: 0; }
.filter-card { margin: 16px 0; }
.notif-card { margin-bottom: 12px; }
.notif-header { display: flex; justify-content: space-between; align-items: center; }
.notif-title { font-weight: 600; font-size: 15px; }
.notif-tags { display: flex; gap: 8px; }
.notif-body { font-size: 14px; color: var(--text-secondary); }
.impact-section { margin-top: 12px; padding: 8px 12px; background: var(--bg-secondary); border-radius: var(--radius-sm); }
.impact-section h4 { margin: 0 0 8px; font-size: 13px; color: var(--text-secondary); }
.impact-item { display: flex; justify-content: space-between; font-size: 13px; margin-bottom: 4px; }
.impact-label { color: var(--text-tertiary); }
.impact-value { font-weight: 600; color: var(--accent-primary); }
.notif-time { margin-top: 8px; font-size: 12px; color: var(--text-tertiary); }
.notif-actions { display: flex; gap: 8px; }
.loading { text-align: center; padding: 40px; color: var(--text-tertiary); }
.change-preview, .impact-preview { background: var(--bg-secondary); padding: 12px; border-radius: var(--radius-sm); font-size: 13px; m…
```

```
.header-card { margin-bottom: 16px; }
</style>
// ======================================== 文件: engineering/src/views/WorkflowPanel.vue ========================================
<template>
  <!-- TODO: 巨型组件拆分 — 工作流编排功能已拆分为 WorkflowPageHeader / WorkflowListPanel / WorkflowDag / WorkflowEventLog / Workflo…
  <div class="workflow-panel-page">
    <!-- ===== Page Header ===== -->
    <WorkflowPageHeader
      :loading="loading"
      :can-cancel="canCancel"
      :can-resume="canResume"
      :current-run-id="currentRunId"
      @refresh="handleRefresh"
      @open-submit="openSubmitDialog"
      @cancel-current="handleCancelCurrent"
      @open-resume="openResumeDialog"
      @delete-current="handleDeleteCurrent"
    />

    <!-- ===== Main Layout: List | DAG + Events ===== -->
    <div class="workflow-main">
      <!-- ===== Left: Workflow List ===== -->
      <WorkflowListPanel
        :workflows="workflows"
        :loading="loading"
        :status-filter="statusFilter"
        :status-options="statusOptions"
        :current-page="currentPage"
        :page-size="pageSize"
        :total-count="totalCount"
        :current-run-id="currentRunId"
        @update:status-filter="handleFilterChange"
        @select="handleSelectWorkflow"
        @update:current-page="handlePageChange"
      />

      <!-- ===== Right: DAG Visualization + Event Log ===== -->
      <!-- TODO: 已拆分到 WorkflowDag / WorkflowEventLog 子组件 -->
      <div class="workflow-detail-panel">
        <WorkflowDag
          :nodes="dagLayout.nodes"
          :edges="dagLayout.edges"
          :width="dagLayout.width"
          :height="dagLayout.height"
          :selected-node-id="selectedNodeId"
          :node-width="nodeWidth"
          :node-height="nodeHeight"
          :spec="currentSpec"
          :title="t('workflowPanel.dagTitle')"
          :empty-text="t('workflowPanel.emptyNoSelection')"
```

```
          :current-display-status="currentDisplayStatus"
          :stream-status-text="streamStatusText"
          :is-stream-connected="stream.isConnected.value"
          :is-stream-done="stream.isDone.value"
          :current-run-id="currentRunId"
          :node-statuses="nodeStatusMap"
          @update:selected-node-id="selectedNodeId = $event"
          @node-click="handleNodeClick"
        />

        <WorkflowEventLog
          :events="stream.events.value"
          :title="t('workflowPanel.eventLogTitle')"
          :empty-text="t('workflowPanel.emptyNoEvents')"
          :btn-clear-text="t('workflowPanel.btnClearEvents')"
          @clear="stream.reset"
        />
      </div>
    </div>

    <!-- ===== Submit / Resume Dialog ===== -->
    <!-- TODO: 已拆分到 WorkflowSubmitDialog 子组件 -->
    <WorkflowSubmitDialog
      :visible="submitDialogVisible"
      :mode="submitMode"
      :title="submitDialogTitle"
      :confirm-button-text="submitConfirmButtonText"
      :form="submitForm"
      :builtin-templates="builtinTemplates"
      :validating="validating"
      :submitting="submitting"
      :form-template-label="t('workflowPanel.formTemplateName')"
      :form-template-placeholder="t('workflowPanel.formTemplatePlaceholder')"
      :form-spec-label="t('workflowPanel.formSpec')"
      :form-spec-placeholder="t('workflowPanel.formSpecPlaceholder')"
      :form-owner-label="t('workflowPanel.formOwnerId')"
      :form-owner-placeholder="t('workflowPanel.formOwnerPlaceholder')"
      :btn-cancel-text="t('workflowPanel.btnCancelDialog')"
      :btn-validate-text="t('workflowPanel.btnValidate')"
      @update:visible="submitDialogVisible = $event"
      @submit="handleSubmit"
      @cancel="submitDialogVisible = false"
      @validate="handleValidate"
      @template-select="handleTemplateSelect"
      @update:form="submitForm = $event"
    />
  </div>
</template>

<script setup lang="ts">
```

```
import { ref, computed, onMounted, watch } from 'vue'
import { useI18n } from 'vue-i18n'
import { ElMessage, ElMessageBox } from 'element-plus'
import { useWorkflow } from '@/composables/useWorkflow'
// 子组件
import WorkflowPageHeader from '@/components/workflow/WorkflowPageHeader.vue'
import WorkflowListPanel from '@/components/workflow/WorkflowListPanel.vue'
import WorkflowDag from '@/components/workflow/WorkflowDag.vue'
import WorkflowEventLog from '@/components/workflow/WorkflowEventLog.vue'
import WorkflowSubmitDialog from '@/components/workflow/WorkflowSubmitDialog.vue'
import type { WorkflowSpec, TaskStatus } from '@/contracts/task'
import { useDagLayout } from '@/composables/useDagLayout'

const { t } = useI18n()

// ---------------------------------------------------------------------------
// useWorkflow composable 接入
// ---------------------------------------------------------------------------
const {
  workflows,
  loading,
  totalCount,
  currentPage,
  pageSize,
  statusFilter,
  loadWorkflows,
  removeWorkflow,
  currentRunId,
  currentStatus,
  submitWorkflow,
  resumeCurrentWorkflow,
  cancelCurrent,
  refreshCurrentStatus,
  selectWorkflow,
  stream,
  validate,
} = useWorkflow()

// ---------------------------------------------------------------------------
// 状态枚举
// ---------------------------------------------------------------------------
const statusOptions = [
  { value: 'pending', label: t('workflowPanel.statusPending') },
  { value: 'queued', label: t('workflowPanel.statusQueued') },
  { value: 'running', label: t('workflowPanel.statusRunning') },
  { value: 'completed', label: t('workflowPanel.statusCompleted') },
  { value: 'failed', label: t('workflowPanel.statusFailed') },
  { value: 'cancelled', label: t('workflowPanel.statusCancelled') },
  { value: 'skipped', label: t('workflowPanel.statusSkipped') },
]
```

```
// ---------------------------------------------------------------------------
// 内置模板（与后端 python/app/workflow/templates/builtin/*.yaml 对应）
// 提供下拉选择，用户选择后填充 spec 编辑器
// ---------------------------------------------------------------------------
const builtinTemplates = ref<Array<{ name: string; version: string; spec: WorkflowSpec }>>([])

const SAMPLE_TOOL_WEAR_SPEC: WorkflowSpec = JSON.parse('{"name": "刀具磨损预测流水线", "version": "1.0.0", "nodes": [{"node_id": "load_dataset…

function initBuiltinTemplates() {
  builtinTemplates.value = [
    { name: SAMPLE_TOOL_WEAR_SPEC.name, version: SAMPLE_TOOL_WEAR_SPEC.version, spec: SAMPLE_TOOL_WEAR_SPEC },
  ]
}

// ---------------------------------------------------------------------------
// 当前选中的工作流 spec（用于 DAG 可视化）
// ---------------------------------------------------------------------------
const currentSpec = computed<WorkflowSpec | null>(() => {
  if (!currentRunId.value) return null
  // 优先从列表中取 spec（完整结构），否则从 currentStatus 取
  const wf = workflows.value.find(w => w.id === currentRunId.value)
  if (wf?.spec) return wf.spec
  return null
})

const currentDisplayStatus = computed<string>(() => {
  // 优先采用 SSE 实时状态，其次持久化状态
  if (stream.currentStatus.value) return stream.currentStatus.value
  if (currentStatus.value?.status) return currentStatus.value.status
  const wf = workflows.value.find(w => w.id === currentRunId.value)
  return wf?.status ?? 'pending'
})

const canCancel = computed(() => {
  const s = currentDisplayStatus.value
  return Boolean(currentRunId.value && (s === 'running' || s === 'queued' || s === 'pending'))
})

const canResume = computed(() => {
  const s = currentDisplayStatus.value
  return Boolean(currentRunId.value && (s === 'failed' || s === 'cancelled'))
})

const streamStatusText = computed(() => {
  if (stream.isDone.value) return t('workflowPanel.streamDone')
  if (stream.isConnected.value) return t('workflowPanel.streamConnected')
  if (stream.error.value) return t('workflowPanel.streamError')
  return t('workflowPanel.streamIdle')
})
```

```
// ---------------------------------------------------------------------------
// 节点状态：合并 SSE nodeStatuses + 持久化 node_statuses
// ---------------------------------------------------------------------------
// TODO: 合并节点状态映射，供 WorkflowDag 子组件使用
const nodeStatusMap = computed<Record<string, TaskStatus>>(() => {
  const map: Record<string, TaskStatus> = {}
  // SSE 节点状态（高优先级）
  if (stream.nodeStatuses.value) {
    for (const [k, v] of Object.entries(stream.nodeStatuses.value)) {
      map[k] = v as TaskStatus
    }
  }
  // 持久化节点状态（兜底）
  if (currentStatus.value?.node_statuses) {
    for (const [k, v] of Object.entries(currentStatus.value.node_statuses)) {
      if (!(k in map)) map[k] = v as TaskStatus
    }
  }
  return map
})

// ---------------------------------------------------------------------------
// DAG 分层布局（自实现，避免引入 dagre 依赖）
// 算法：Kahn 拓扑排序 + 按入度分层
// ---------------------------------------------------------------------------
const nodeWidth = 160
const nodeHeight = 76

const dagLayout = useDagLayout(() => currentSpec.value)

// 子组件已内置 isEdgeActive / getNodeStatus 逻辑

// ---------------------------------------------------------------------------
// 列表交互
// ---------------------------------------------------------------------------
function handleRefresh() {
  loadWorkflows()
  if (currentRunId.value) refreshCurrentStatus()
}

function handleFilterChange(value: string) {
  statusFilter.value = value
  currentPage.value = 1
  loadWorkflows()
}

function handlePageChange(page: number) {
  currentPage.value = page
  loadWorkflows()
```

```
}

async function handleSelectWorkflow(id: string) {
  try {
    await selectWorkflow(id)
  } catch (e) {
    console.warn('[WorkflowPanel] selectWorkflow failed:', e)
  }
}

async function handleCancelCurrent() {
  if (!currentRunId.value) return
  try {
    await ElMessageBox.confirm(
      t('workflowPanel.confirmCancel'),
      t('workflowPanel.warning'),
      { type: 'warning' },
    )
  } catch {
    return // 用户取消
  }
  try {
    await cancelCurrent()
    ElMessage.success(t('workflowPanel.msgCancelSuccess'))
  } catch (e) {
    console.warn('[WorkflowPanel] cancel failed:', e)
  }
}

async function handleDeleteCurrent() {
  if (!currentRunId.value) return
  try {
    await ElMessageBox.confirm(
      t('workflowPanel.confirmDelete'),
      t('workflowPanel.warning'),
      { type: 'warning' },
    )
  } catch {
    return
  }
  try {
    await removeWorkflow(currentRunId.value)
    ElMessage.success(t('workflowPanel.msgDeleteSuccess'))
  } catch (e) {
    console.warn('[WorkflowPanel] delete failed:', e)
  }
}

// ---------------------------------------------------------------------
// 提交 / 续跑对话框
```

```
// ---------------------------------------------------------------------------
const submitDialogVisible = ref(false)
const submitMode = ref<'submit' | 'resume'>('submit')
const submitForm = ref({
  templateName: '',
  specYaml: '',
  ownerId: '',
})
const validating = ref(false)
const submitting = ref(false)
const selectedNodeId = ref<string>('')

function handleNodeClick(nodeId: string) {
  selectedNodeId.value = nodeId
}

const submitDialogTitle = computed(() =>
  submitMode.value === 'submit'
    ? t('workflowPanel.dialogSubmitTitle')
    : t('workflowPanel.dialogResumeTitle'),
)

const submitConfirmButtonText = computed(() =>
  submitMode.value === 'submit'
    ? t('workflowPanel.btnSubmitConfirm')
    : t('workflowPanel.btnResumeConfirm'),
)

function openSubmitDialog() {
  submitMode.value = 'submit'
  submitForm.value = { templateName: '', specYaml: '', ownerId: '' }
  submitDialogVisible.value = true
}

function openResumeDialog() {
  if (!currentRunId.value) return
  submitMode.value = 'resume'
  // 续跑时预填当前 spec
  const wf = workflows.value.find(w => w.id === currentRunId.value)
  if (wf?.spec) {
    submitForm.value = {
      templateName: '',
      specYaml: JSON.stringify(wf.spec, null, 2),
      ownerId: wf.owner_id ?? '',
    }
  } else {
    submitForm.value = { templateName: '', specYaml: '', ownerId: '' }
  }
  submitDialogVisible.value = true
}
```

```
function handleTemplateSelect(name: string) {
  if (!name) return
  const tpl = builtinTemplates.value.find(t => t.name === name)
  if (tpl) {
    submitForm.value.specYaml = JSON.stringify(tpl.spec, null, 2)
  }
}

function parseSpec(): WorkflowSpec | null {
  const text = submitForm.value.specYaml.trim()
  if (!text) {
    ElMessage.warning(t('workflowPanel.msgSpecEmpty'))
    return null
  }
  try {
    // 后端模板是 YAML，前端为简化依赖使用 JSON 解析；
    // 用户也可粘贴 YAML（需后端 /validate 端点解析，此处先尝试 JSON）
    const obj = JSON.parse(text) as WorkflowSpec
    if (!obj.name || !obj.nodes || !obj.edges) {
      ElMessage.error(t('workflowPanel.msgSpecInvalid'))
      return null
    }
    return obj
  } catch {
    // JSON 解析失败时尝试 YAML 简单解析（key: value 形式）
    ElMessage.error(t('workflowPanel.msgSpecParseError'))
    return null
  }
}

async function handleValidate() {
  const spec = parseSpec()
  if (!spec) return
  validating.value = true
  try {
    const result = await validate(spec)
    if (result.valid) {
      ElMessage.success(
        t('workflowPanel.msgValidateSuccess')
          .replace('{nodes}', String(result.node_count))
          .replace('{edges}', String(result.edge_count)),
      )
    } else {
      ElMessage.warning(t('workflowPanel.msgValidateFailed'))
    }
  } catch (e) {
    console.warn('[WorkflowPanel] validate failed:', e)
  } finally {
    validating.value = false
```

```
  }
}

async function handleSubmit() {
  const spec = parseSpec()
  if (!spec) return
  submitting.value = true
  try {
    if (submitMode.value === 'submit') {
      const runId = await submitWorkflow({
        spec,
        owner_id: submitForm.value.ownerId || undefined,
      })
      ElMessage.success(t('workflowPanel.msgSubmitSuccess').replace('{id}', runId.slice(0, 12)))
      submitDialogVisible.value = false
      await loadWorkflows()
    } else {
      // 续跑：基于当前 run_id
      if (!currentRunId.value) {
        ElMessage.warning(t('workflowPanel.msgNoCurrentRun'))
        return
      }
      const newRunId = await resumeCurrentWorkflow(currentRunId.value, {
        spec,
        owner_id: submitForm.value.ownerId || undefined,
      })
      ElMessage.success(t('workflowPanel.msgResumeSuccess').replace('{id}', newRunId.slice(0, 12)))
      submitDialogVisible.value = false
      await loadWorkflows()
    }
  } catch (e) {
    console.warn('[WorkflowPanel] submit/resume failed:', e)
  } finally {
    submitting.value = false
  }
}

// ---------------------------------------------------------------------------
// 生命周期
// ---------------------------------------------------------------------------
onMounted(() => {
  initBuiltinTemplates()
  loadWorkflows()
})

// 监听 SSE 终态：自动刷新列表与持久化状态
watch(
  () => stream.isDone.value,
  (done) => {
    if (done) {
```

```
      void loadWorkflows()
      if (currentRunId.value) void refreshCurrentStatus()
    }
  },
)
</script>

<style scoped>
/* TODO: 巨型组件拆分 — 样式已迁移到子组件中 */
.workflow-panel-page {
  padding: 16px;
  height: 100%;
  display: flex;
  flex-direction: column;
  gap: 12px;
  overflow: hidden;
}

/* ===== Main Layout ===== */
.workflow-main {
  flex: 1;
  display: grid;
  grid-template-columns: 320px 1fr;
  gap: 12px;
  min-height: 0;
}

/* ===== Detail Panel ===== */
.workflow-detail-panel {
  display: grid;
  grid-template-rows: 1fr 200px;
  gap: 12px;
  min-height: 0;
}
</style>
// ==================================== 文件: engineering/src/views/Workspace.vue ====================================
<template>
  <div class="workspace-page">
    <el-card>
      <template #header>
        <div class="header-with-actions">
          <span>{{ $t('workspace.header') }}</span>
          <el-tag
            type="info"
            size="small"
          >
            {{ $t('workspace.userSovereignty') }}
          </el-tag>
        </div>
      </template>
```

```
      <el-tabs v-model="activeTab">
        <el-tab-pane
          :label="$t('workspace.predictTab')"
          name="predict"
        >
          <WorkspacePredictTab />
        </el-tab-pane>

        <el-tab-pane
          :label="$t('workspace.trainTab')"
          name="train"
        >
          <WorkspaceTrainForm
            :train-form="trainForm"
            :dry-running="dryRunning"
            :training="training"
            :train-plan-confirmed="trainPlanConfirmed"
            @dry-run="handleDryRun"
            @train="handleTrain"
            @update:train-form="Object.assign(trainForm, $event)"
          />
          <WorkspaceTrainMonitor
            :dry-run-result="dryRunResult"
            :train-result="trainResult"
            :current-job-id="currentJobId"
          :sse="{ currentStatus: sse.currentStatus ?? null, progress: sse.progress ?? 0, lastProgressData: (sse.lastProgressData ?? …
            :loss-history="lossHistory"
            :val-loss-history="valLossHistory"
            :cancelling="cancelling"
            :train-plan-confirmed="trainPlanConfirmed"
            @cancel-training="handleCancelTraining"
            @update:train-plan-confirmed="trainPlanConfirmed = $event"
          />
        </el-tab-pane>

        <el-tab-pane
          :label="$t('workspace.modelsTab')"
          name="models"
        >
          <WorkspaceModelsTab />
        </el-tab-pane>
      </el-tabs>
    </el-card>
  </div>
</template>

<script setup lang="ts">
// TODO(P1-3): 巨型组件拆分 — 本文件 1087 行，应拆分为子组件/composable：
//   - 工作区列表/网格 → WorkspaceGrid.vue
//   - 项目卡片 → ProjectCard.vue
```

```
//   - 创建/编辑弹窗 → ProjectDialog.vue
//   - 数据获取逻辑 → useWorkspace.ts
// 拆分时注意保持 props/emits 接口不变，逐模块迁移并验证。
import { ref, reactive, onMounted, computed } from 'vue'
import { useI18n } from 'vue-i18n'
import { ElMessage, ElMessageBox } from 'element-plus'
import http from '@/utils/http'
import WorkspacePredictTab from '@/components/workspace/WorkspacePredictTab.vue'
import WorkspaceModelsTab from '@/components/workspace/WorkspaceModelsTab.vue'
import WorkspaceTrainForm from '@/components/workspace/WorkspaceTrainForm.vue'
import WorkspaceTrainMonitor from '@/components/workspace/WorkspaceTrainMonitor.vue'
import { useEventSource } from '@/composables/useEventSource'
import { API_CONFIG, buildApiPath } from '@/config/api'

const { t } = useI18n()
const activeTab = ref('predict')
const training = ref(false)
const dryRunning = ref(false)
const trainPlanConfirmed = ref(false)

interface DryRunResult {
  is_dry_run: boolean
  training_plan: {
    estimated_duration_minutes: number
    estimated_memory_mb: number
    estimated_gpu_memory_mb?: number
    dataset_samples: number
    train_val_split: { train: number; validation: number; ratio: string }
    potential_risks: string[]
    recommendations: string[]
  }
  confidence: number
  reasoning: string
}

interface TrainResult {
  job_id: string
  status: string
  message?: string
}

interface ModelInfo {
  name: string
  model_type: string
  version: string
  input_features?: string[]
}

const trainForm = reactive({
  modelName: '',
```

```
  dataPath: '',
  hyperparameters: {
    learning_rate: 0.001,
    epochs: 100,
    batch_size: 32,
    optimizer: 'adam',
  },
  device: 'auto',
})

const dryRunResult = ref<DryRunResult | null>(null)
const trainResult = ref<TrainResult | null>(null)
const modelList = ref<ModelInfo[]>([])

const currentJobId = ref<string | null>(null)
const sseJobId = ref('')
const sse = reactive(useEventSource(sseJobId, { autoReconnect: true, maxRetries: 10 }))
const cancelling = ref(false)

function connectToJob(jobId: string) {
  sseJobId.value = jobId
  sse.reset()
  sse.connect()
}

const lossHistory = computed(() => {
  const losses: number[] = []
  if (!sse.events) return losses
  for (const event of sse.events) {
    if (event.type === 'progress' && event.data.metrics?.train_loss !== undefined) {
      losses.push(event.data.metrics.train_loss as number)
    }
  }
  return losses
})

const valLossHistory = computed(() => {
  const losses: number[] = []
  if (!sse.events) return losses
  for (const event of sse.events) {
    if (event.type === 'progress' && event.data.metrics?.val_loss !== undefined) {
      losses.push(event.data.metrics.val_loss as number)
    }
  }
  return losses
})

async function handleDryRun() {
  if (!trainForm.modelName || !trainForm.dataPath) {
    ElMessage.warning(t('common.inputPlaceholder'))
```

```
    return
  }

  dryRunning.value = true
  dryRunResult.value = null
  trainPlanConfirmed.value = false

  try {
    const res = await http.post(buildApiPath(API_CONFIG.LNN, '/train/dry_run'), {
      model_name: trainForm.modelName,
      data_path: trainForm.dataPath,
      hyperparameters: trainForm.hyperparameters,
      device: trainForm.device,
    })

    dryRunResult.value = res.data.data as DryRunResult
    ElMessage.success(t('workspace.trainingPlanSummary'))
  } catch (e: unknown) {
    const errorMsg = e instanceof Error ? e.message : String(e)
    ElMessage.error(errorMsg || t('common.unknownError'))
  } finally {
    dryRunning.value = false
  }
}

async function handleTrain() {
  if (!trainPlanConfirmed.value) {
    ElMessage.warning(t('workspace.confirmTraining'))
    return
  }

  training.value = true
  trainResult.value = null

  try {
    const res = await http.post(buildApiPath(API_CONFIG.LNN, '/train'), {
      model_name: trainForm.modelName,
      data_path: trainForm.dataPath,
      hyperparameters: trainForm.hyperparameters,
      device: trainForm.device,
    })

    const jobId = res.data.data?.job_id
    if (!jobId) {
      ElMessage.error(t('workspace.jobId'))
      return
    }

    currentJobId.value = jobId
    connectToJob(jobId)
```

```
    trainResult.value = res.data.data
    ElMessage.success(t('workspace.trainingMonitor'))

    await recordAuditLog('lnn_train', dryRunResult.value, 'accept', 'success', trainForm)
  } catch (e: unknown) {
    const errorMsg = e instanceof Error ? e.message : String(e)
    ElMessage.error(errorMsg || t('common.unknownError'))
    await recordAuditLog('lnn_train', dryRunResult.value, 'reject', 'failed', trainForm)
  } finally {
    training.value = false
  }
}

async function handleCancelTraining() {
  if (!currentJobId.value) return

  try {
    await ElMessageBox.confirm(t('workspace.confirmCancelTraining'), t('workspace.confirmCancelTitle'), {
      confirmButtonText: t('common.confirm'),
      cancelButtonText: t('common.cancel'),
      type: 'warning',
    })

    cancelling.value = true
    await http.post(buildApiPath(API_CONFIG.JOBS, `/${currentJobId.value}/cancel`))
    ElMessage.info(t('workspace.trainingCancelled'))
  } catch (e: unknown) {
    if (e !== 'cancel') {
      ElMessage.error(t('common.failed'))
    }
  } finally {
    cancelling.value = false
  }
}

async function recordAuditLog(
  aiModule: string,
  aiRecommendation: Record<string, unknown> | null,
  userDecision: string,
  operationStatus: string,
  finalExecution?: Record<string, unknown>,
) {
  try {
    await http.post(buildApiPath(API_CONFIG.USER_SOVEREIGNTY, '/audit-log/record'), null, {
      params: {
        ai_module: aiModule,
        ai_recommendation: JSON.stringify(aiRecommendation || {}),
        user_decision: userDecision,
        final_execution: JSON.stringify(finalExecution || {}),
```

```
        operation_status: operationStatus,
        confidence: dryRunResult.value?.confidence || null,
        reasoning: dryRunResult.value?.reasoning || null,
      },
    })
  } catch (e: unknown) {
    // 审计日志记录失败不应阻塞用户主流程，但需记录便于后续审计追溯
    console.warn('[Workspace] recordAuditLog failed:', e)
  }
}

onMounted(async () => {
  try {
    const res = await http.get(buildApiPath(API_CONFIG.LNN, '/models'))
    modelList.value = res.data?.data?.models || []
  } catch {
    modelList.value = []
  }
})
</script>

<style scoped>
.workspace-page {
  max-width: 1200px;
  margin: 0 auto;
}

.header-with-actions {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.result-section {
  background: var(--bg-secondary);
  border-radius: var(--radius-md);
  padding: 16px;
  border: 1px solid var(--border-light);
}

.result-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 16px;
}

.prediction-value {
  margin: 16px 0;
  padding: 12px;
```

```
  background: var(--bg-card);
  border-radius: var(--radius-sm);
  border-left: 4px solid var(--accent-primary);
}

.prediction-value .label {
  font-weight: 600;
  color: var(--text-secondary);
  margin-right: 8px;
}

.prediction-value .value {
  font-size: 18px;
  font-weight: 700;
  color: var(--text-primary);
}

.reasoning-section {
  margin: 16px 0;
  padding: 12px;
  background: var(--bg-card);
  border-radius: var(--radius-sm);
  border-left: 4px solid var(--success);
}

.reasoning-section h5 {
  margin: 0 0 8px 0;
  color: var(--success);
  font-weight: 600;
}

.reasoning-section p {
  margin: 0;
  color: var(--text-secondary);
  line-height: 1.6;
}

.adjusted-result {
  margin-top: 16px;
  padding: 12px;
  background: var(--bg-card);
  border-radius: var(--radius-sm);
  border: 1px solid var(--border-light);
}

.adjusted-result pre {
  margin: 0;
  white-space: pre-wrap;
  word-break: break-all;
  font-size: 12px;
```

```
  color: var(--text-secondary);
}
</style>
// ==================================== 文件: engineering/src/views/WorldModel.vue ====================================
<!--
  世界模型视图（ADR-017 阶段 8 p8-7）

  对应后端 `python/app/api/v1/world_model.py`。
  详见 `docs/adr/ADR-017-世界模型与RL模块.md`。

  功能：
    1. 世界模型版本列表（左侧）+ 版本详情（右侧顶部）
    2. 轨迹预测面板：输入 current_state + candidate_action + horizon，
       调用 store.predict()，展示预测轨迹与汇总指标
    3. 工程约束：v1 仅离线预测，结果供 RL agent 训练参考，
       不直接接 CNC 控制器（物理执行需持证操作员 + 导师签字 + 保险）
-->
<template>
  <div class="world-model-page">
    <!-- ===== 页面头部 ===== -->
    <header class="page-header">
      <div class="page-header__title-block">
        <h1 class="page-header__title">{{ t('worldModel.title') }}</h1>
        <p class="page-header__subtitle">{{ t('worldModel.subtitle') }}</p>
      </div>
      <div class="page-header__actions">
        <el-button :icon="Refresh" @click="handleRefresh" :loading="store.anyLoading">
          {{ t('common.refresh') }}
        </el-button>
      </div>
    </header>

    <!-- ===== 主体：左列表 + 右详情/预测 ===== -->
    <div class="wm-main">
      <!-- 左侧：版本列表 -->
      <WorldModelVersionList
        :versions="store.versions"
        :current-version="store.currentVersion"
        :versions-loading="store.versionsLoading"
        :total-pages="store.totalPages"
        :version-pagination="store.versionPagination"
        :current-page="currentPage"
        @select-version="handleSelectVersion"
        @update:current-page="handlePageChange"
      />

      <!-- 右侧：版本详情 + 预测面板 -->
      <section class="wm-detail-panel">
        <!-- 版本详情 -->
        <WorldModelVersionDetail
```

```
          :current-version="store.currentVersion"
          :version-loading="store.versionLoading"
          @use-active-version="handleUseActiveVersion"
        />

        <!-- 预测面板 -->
        <WorldModelPredictPanel
          :model-uri="predictForm.model_uri"
          :horizon="predictForm.horizon"
          :current-state="predictForm.current_state"
          :candidate-action="predictForm.candidate_action"
          :predicting="store.predicting"
          :last-prediction="store.lastPrediction"
          :last-prediction-max-chatter="store.lastPredictionMaxChatter"
          :last-prediction-step-count="store.lastPredictionStepCount"
          @update:model-uri="predictForm.model_uri = $event"
          @update:horizon="predictForm.horizon = $event"
          @update-state="handleUpdateState"
          @update-action="handleUpdateAction"
          @predict="handlePredict"
          @reset-form="handleResetForm"
        />
      </section>
    </div>
  </div>
</template>

<script setup lang="ts">
import { onMounted, reactive, ref } from 'vue'
import { useI18n } from 'vue-i18n'
import { ElMessage } from 'element-plus'
import { Refresh } from '@element-plus/icons-vue'
import { useWorldModelStore } from '@/stores/worldModel'
import {
  STATE_FIELD,
  ACTION_FIELD,
  DEFAULT_HORIZON,
  DEFAULT_WORLD_MODEL_URI,
  type WorldModelPredictRequest,
} from '@/contracts/world_model'
import WorldModelVersionList from '@/components/world_model/WorldModelVersionList.vue'
import WorldModelVersionDetail from '@/components/world_model/WorldModelVersionDetail.vue'
import WorldModelPredictPanel from '@/components/world_model/WorldModelPredictPanel.vue'

const { t } = useI18n()
const store = useWorldModelStore()

const currentPage = ref(1)

/** 默认状态值（典型 6061-T6 粗加工场景） */
```

```
const defaultStateValue: Record<string, number> = {
  [STATE_FIELD.SPINDLE_SPEED]: 8000,
  [STATE_FIELD.FEED_RATE]: 1200,
  [STATE_FIELD.DEPTH_OF_CUT]: 1.5,
  [STATE_FIELD.WIDTH_OF_CUT]: 6.0,
  [STATE_FIELD.TOOL_WEAR]: 0.05,
  [STATE_FIELD.VIBRATION_RMS]: 0.8,
  [STATE_FIELD.TEMPERATURE]: 45.0,
  [STATE_FIELD.CHATTER_PROBABILITY]: 0.1,
}

const defaultActionValue: Record<string, number> = {
  [ACTION_FIELD.SPINDLE_SPEED_DELTA]: 0.0,
  [ACTION_FIELD.FEED_RATE_DELTA]: 0.0,
  [ACTION_FIELD.DEPTH_OF_CUT_DELTA]: 0.0,
  [ACTION_FIELD.WIDTH_OF_CUT_DELTA]: 0.0,
}

const predictForm = reactive<{
  model_uri: string
  horizon: number
  current_state: Record<string, number>
  candidate_action: Record<string, number>
}>({
  model_uri: DEFAULT_WORLD_MODEL_URI,
  horizon: DEFAULT_HORIZON,
  current_state: { ...defaultStateValue },
  candidate_action: { ...defaultActionValue },
})

/** 处理状态字段更新 */
function handleUpdateState(field: string, value: number): void {
  predictForm.current_state[field] = value
}

/** 处理动作字段更新 */
function handleUpdateAction(field: string, value: number): void {
  predictForm.candidate_action[field] = value
}

/** 加载版本列表 */
async function loadVersions(): Promise<void> {
  const result = await store.fetchVersions({ limit: 50, offset: (currentPage.value - 1) * 50 })
  if (!result) {
    ElMessage.error(store.error || t('worldModel.loadFailed'))
  }
}

/** 选择版本查看详情 */
async function handleSelectVersion(version: string): Promise<void> {
```

```
  const result = await store.fetchVersion(version)
  if (!result) {
    ElMessage.error(store.error || t('worldModel.loadFailed'))
  }
}

/** 使用当前版本填充预测表单 model_uri */
function handleUseActiveVersion(): void {
  if (store.currentVersion) {
    predictForm.model_uri = store.currentVersion.model_uri
    ElMessage.success(t('worldModel.modelUriFilled'))
  }
}

/** 翻页 */
function handlePageChange(page: number): void {
  currentPage.value = page
  void loadVersions()
}

/** 刷新 */
async function handleRefresh(): Promise<void> {
  currentPage.value = 1
  await loadVersions()
}

/** 执行预测 */
async function handlePredict(): Promise<void> {
  const request: WorldModelPredictRequest = {
    model_uri: predictForm.model_uri,
    horizon: predictForm.horizon,
    current_state: { ...predictForm.current_state },
    candidate_action: { ...predictForm.candidate_action },
  }
  const result = await store.predict(request)
  if (!result) {
    ElMessage.error(store.error || t('worldModel.predictFailed'))
  } else {
    ElMessage.success(t('worldModel.predictSuccess'))
  }
}

/** 重置表单 */
function handleResetForm(): void {
  predictForm.model_uri = DEFAULT_WORLD_MODEL_URI
  predictForm.horizon = DEFAULT_HORIZON
  predictForm.current_state = { ...defaultStateValue }
  predictForm.candidate_action = { ...defaultActionValue }
  store.clearLastPrediction()
}
```

```
onMounted(() => {
  void loadVersions()
})
</script>

<style scoped>
.world-model-page {
  padding: 20px;
  display: flex;
  flex-direction: column;
  gap: 16px;
}

/* ===== 页面头部 ===== */
.page-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  gap: 16px;
}

.page-header__title {
  margin: 0;
  font-size: 22px;
  font-weight: 600;
  color: var(--el-text-color-primary);
}

.page-header__subtitle {
  margin: 4px 0 0;
  font-size: 13px;
  color: var(--el-text-color-secondary);
}

/* ===== 主体布局 ===== */
.wm-main {
  display: grid;
  grid-template-columns: 360px 1fr;
  gap: 16px;
  align-items: start;
}

/* ===== 右侧详情 ===== */
.wm-detail-panel {
  display: flex;
  flex-direction: column;
  gap: 16px;
}
</style>
```